

作業ログ

報告者：山口汐里

報告日：2026 年 6 月 9 日

プロジェクト名：受動的環境介入によるメンタルヘルス支援：大学空間における自動演奏装置の提案

1. 進捗サマリー

- 全体進捗率：75%（音声合成フェーズ収束・通信フェーズ開始）
 - マイルストーン達成状況：進行中
 - 主要成果：
 - flute4.pde にメロディ演奏機能（7 音符）を追加し、フルート音色で連続演奏を確認
 - flute5.pde で呼気ノイズを 2 系統（2000 Hz 帯・700 Hz 帯）に分割し、音色の自然さを改善
 - Arduino I2C 通信の基本実装（simpleI2C）でマスター/スレーブ間ハンドシェイクを確認
 - BPM 同期付き I2C 通信（sync-I2C）で CMD_SYNC / CMD_START / CMD_CONFIG コマンド体系を実装
 - 重大課題（影響度：高）：
 - なし
-

2. プロジェクト計画（ガントチャート）

図 1 にプロジェクト全体のガントチャートを示す。今週（5/27～6/9）は WBS 2.1.3「フルート音色改良」および WBS 3.1「I2C 通信基盤」に相当し、第 7～10 週の前半にあたる。本期間の作業はスケジュール通りに完了した。



図1 Arduino オーケストラ プロジェクトガントチャート

3. 作業ログ

作業項目	開始日	予定完了	状態	進捗率	コメント
flute4 実装 (メロディ演奏機能追加)	5/27	5/27	完了	100%	flute3 にメロディ配列を追加, 7 音符 (きらき星冒頭) で連続演奏を確認
flute4 聴感・動作テスト	5/28	5/28	完了	100%	メロディ演奏時の音質確認, 前回 (flute3) との比較で問題なし
flute5 設計 (呼気ノイズ 2 系統化)	5/28	5/29	完了	100%	breathEnv2 (700 Hz 帯バンドパス) 追加の設計, ウェーブテーブル 10 倍音化を検討
flute5 実装	5/29	5/29	完了	100%	breathEnv2 実装, ウェーブテーブルを 10 倍音に拡張, ビブラートを 15 セントに強化
flute5 聴感テスト・調整	5/30	5/30	完了	100%	単音 C4 (5 秒) で各パラメータを確認, 2 系統ノイズによる音色改善を確認

作業項目	開始日	予定完了	状態	進捗率	コメント
simpleI2C 実装	6/2	6/2	完了	100%	Arduino 間 I2C 基本通信（マスター/スレーブ）を実装，1 バイト送受信で疎通確認
simpleI2C 動作確認	6/3	6/3	完了	100%	シリアルモニタでエコー応答バイトを確認，正常に受信できることを検証
sync-I2C 実装	6/4	6/5	完了	100%	CMD_SYNC / CMD_START / CMD_CONFIG コマンド体系を設計・実装，ISR 内制約に対処
sync-I2C 動作確認	6/5	6/6	完了	100%	BPM 同期付き通信の動作をシリアルモニタで検証，bar カウントと bpmX10 の受信を確認
作業ログ作成	6/9	6/9	完了	100%	本ドキュメントの作成

4. 課題管理

課題	発生原因	影響度	優先度	対策	期限	状態
ISR 内での Serial.print 使用不可	Wire.onReceive コールバックが割り込みサービスルーチン (ISR) 内で実行されるため，Serial.print が不安定になる	中	高	フラグ変数を ISR 内でセットし，メインループ側で出力する方式に変更	6/5	解決
高次倍音の精度不足	実測 FFT 値とウェーブテーブル係数の微小な差異	低	低	次フェーズで係数をさらに調整	未定	対応中

5. AI 利用状況と検証

5.1 利用内容

- 利用目的：設計支援・実装支援
- 入力（プロンプト要旨）：
 - flute5 における呼気ノイズ 2 系統化（2000 Hz 帯 + 700 Hz 帯）の設計方針を相談
 - Arduino I2C の CMD 体系（CMD_SYNC / CMD_START / CMD_CONFIG）の設計方針を相談

- `Wire.onReceive` が ISR 内で実行される際の制約と対処方法を質問
 - 出力概要：
 - 700 Hz 帯の呼気ノイズを追加することで低域の息の成分を再現できるという方針を得た
 - コマンド種別を 1 バイト目に配置し、後続バイト数をコマンドごとに固定する設計を提案された
 - ISR 内では `Serial.print` を使わず、フラグ変数をセットしてメインループで処理する方式を確認した
-

5.2 検証方法

- 検証手法：
 - flute5：生成音声とサンプル音声の聴感比較，WavePad でのスペクトル目視確認
 - sync-I2C：Arduino シリアルモニタでの受信データ目視確認
 - 参照資料：
 - Arduino I2C 通信の実装 [1]
 - 複数 Arduino のシリアル通信制御 [2]
 - 検証手順（sync-I2C）：
 1. マスター Arduino と スレーブ Arduino を I2C で接続（SDA / SCL / GND）
 2. マスター側スケッチを書き込み，シリアルモニタを開く
 3. スレーブ側スケッチを書き込み，受信ログをシリアルモニタで確認
 4. CMD_SYNC 受信時に bar カウントと bpmX10 が正しく出力されることを目視確認
 5. CMD_CONFIG 受信時に entry_offset と loop_length が正しく設定されることを確認
-

5.3 ファクトチェック結果

- 一致点：
 - I2C コマンド体系（1 バイト目にコマンド種別，後続バイトにペイロード）は参考文献 [1] で紹介されている基本的な送受信パターンと整合
 - マスター/スレーブ構成でのシリアル通信制御の実装方式は参考文献 [2] のアーキテクチャと一致
 - 根拠：
 - シリアルモニタでの受信データ確認，および flute5 の聴感テスト結果
-

6. 次回計画

- 目標：
 - Processing + Minim による `flute5_serial` の実装（シリアル受信 → 即時再生）
 - 複数スレーブ対応マスター（SYNCs）への拡張

- 達成基準：
 - Arduino から送信されたバイナリプロトコルを Processing が受信し、ノイズなく即時再生できること
 - 3 台のスレーブに CMD_CONFIG / CMD_START / CMD_SYNC を送信し、各スレーブが独立して動作すること
- 期限：
 - 未定

7. 評価基準

今回の評価指標を以下に示す。現時点では音声合成フェーズが収束しつつあり、通信フェーズの基盤が整った段階である。評価テストは第 8～10 週に実施予定。

WBS	評価項目	担当	実施時期	達成基準
3.1	単体・周期精度テスト (V1 音程 / V2 FFT 比較)	全員	第 8～9 週	スペクトル目視一致・音程誤差許容範囲内
3.2	全体合奏演奏テスト	全員	第 9～10 週	合奏時間 120 秒以上の連続動作
3.3	可変テンポ動作検証	全員	第 10 週	テンポ誤差 ± 50 ms 以内

本週 (WBS 2.1.3 / 3.1) に対する自己評価：

- flute5 では呼気ノイズを 2 系統に分割し、聴感テストでフルートらしさの改善を確認した
- I2C 通信の基本実装から BPM 同期付き通信まで段階的に実装でき、コマンド体系の設計指針を確立した
- ISR 内制約 (Serial.print 禁止) は同期設計の重要な制約として把握・対処済み

8. その他

- 使用ソフトウェア：Arduino IDE (I2C 実装・シリアルモニタ確認)、Processing + Minim (音声合成テスト)
- WavePad によるスペクトル比較スクリーンショットはプロジェクトルートに保存済み

9. 付録

- 添付範囲：
 - flute4.pde：flute3 からの変更点 (メロディ配列追加) が主体
 - flute5.pde：flute4 からの変更点 (呼気ノイズ 2 系統化・倍音拡張) が主体
 - simpleI2C / sync-I2C：全文を掲載

Listing 1 flute4.pde

```

1  import ddf.minim.*;
2  import ddf.minim.ugens.*;
3
4  Minim minim;
5  AudioOutput out;
6  Waveform currentWaveform;
7
8  String[] melody = {
9    "C4", "C4", "G4", "G4", "A4", "A4", "G4"
10 };
11
12 float[] duration = {
13   2.0f, 2.0f, 2.0f, 2.0f, 2.0f, 2.0f, 2.0f
14 };
15
16 float[] startTime = {
17   0.0f, 2.0f, 4.0f, 6.0f, 8.0f, 10.0f, 12.0f
18 };
19
20 class FluteInstrument implements Instrument
21 {
22   ADSR toneEnv;
23   ADSR breathEnv;
24
25   FluteInstrument(float frequency, float maxAmp, Waveform wf)
26   {
27     float cutoff = (float)Math.min(9000.0, frequency * 6.0);
28
29     Oscil tone = new Oscil(frequency, 1.0f, wf);
30
31     float vibrateSpeed = 4.3f;
32     float vibratoSize = 7.4f;
33     float vibratoNow = frequency * (pow(2, vibratoSize / 1200.0f) - 1.0f);
34
35     Oscil vibrato = new Oscil(vibrateSpeed, vibratoNow, Waves.SINE);
36     vibrato.offset.setLastValue(frequency);
37     vibrato.patch(tone.frequency);
38
39     MoogFilter toneLPF = new MoogFilter(cutoff, 0.0f, MoogFilter.Type.LP);
40
41     Noise breath = new Noise(1.0f, Noise.Tint.WHITE);
42     MoogFilter breathBand = new MoogFilter(2000.0f, 0.15f, MoogFilter.Type.BP);
43
44     toneEnv = new ADSR(maxAmp, 0.06f, 0.14f, 0.82f, 0.16f);
45     breathEnv = new ADSR(maxAmp * 0.08f, 0.015f, 0.12f, 0.04f, 0.10f);
46
47     tone.patch(toneLPF);
48     toneLPF.patch(toneEnv);
49
50     breath.patch(breathBand);
51     breathBand.patch(breathEnv);
52   }

```

```

53
54 void noteOn(float duration)
55 {
56     toneEnv.patch(out);
57     breathEnv.patch(out);
58
59     toneEnv.unpatchAfterRelease(out);
60     breathEnv.unpatchAfterRelease(out);
61
62     toneEnv.noteOn();
63     breathEnv.noteOn();
64 }
65
66 void noteOff()
67 {
68     toneEnv.noteOff();
69     breathEnv.noteOff();
70 }
71 }
72
73
74 void setup()
75 {
76     size(512, 220);
77     minim = new Minim(this);
78     out = minim.getLineOut();
79     out.setTempo(120);
80
81     currentWaveform = makeFluteWaveform();
82 }
83
84 Wavetable makeFluteWaveform()
85 {
86     Wavetable wf = WavetableGenerator.gen10(
87         8192,
88         new float[] {
89             1.00f, 0.35f, 0.14f, 0.12f,
90             0.08f, 0.04f, 0.025f, 0.02f
91         }
92     );
93
94     wf.normalize();
95     wf.scale(0.75f);
96     return wf;
97 }
98
99 void playSong()
100 {
101     out.pauseNotes();
102
103     for (int i = 0; i < melody.length; i++)
104     {
105         out.playNote(
106             startTime[i],

```

```

107     duration[i],
108     new FluteInstrument(
109         Frequency.ofPitch( melody[i] ).asHz(), 0.45f, currentWaveform)
110     );
111 }
112
113 out.resumeNotes();
114 }
115
116 void draw()
117 {
118     background(0);
119     stroke(255);
120
121     for (int i = 0; i < out.bufferSize() - 1; i++)
122     {
123         line(i, 70 - out.left.get(i) * 60,
124             i + 1, 70 - out.left.get(i + 1) * 60);
125
126         line(i, 160 - out.right.get(i) * 60,
127             i + 1, 160 - out.right.get(i + 1) * 60);
128     }
129
130     fill(255);
131
132 }
133
134 void keyPressed()
135 {
136     switch(key)
137     {
138     case 'p':
139         playSong();
140         break;
141
142     case '1':
143         currentWaveform = makeFluteWaveform();
144         break;
145
146     }
147 }
148
149 void stop()
150 {
151     minim.stop();
152     super.stop();
153 }

```

Listing 2 flute5.pde

```

1 import ddf.minim.*;
2 import ddf.minim.ugens.*;
3

```



```

4 Minim minim;
5 AudioOutput out;
6 Waveform currentWaveform;
7
8 String[] melody = {
9     "C4"
10 };
11
12 float[] duration = {
13     5.0f
14 };
15
16 float[] startTime = {
17     0.0f
18 };
19
20 class FluteInstrument implements Instrument
21 {
22     ADSR toneEnv;
23     ADSR breathEnv;
24     ADSR breathEnv2;
25
26     FluteInstrument(float frequency, float maxAmp, Waveform wf)
27     {
28         float cutoff = (float)Math.min(9000.0, frequency * 9.0);
29
30         Oscil tone = new Oscil(frequency, 1.0f, wf);
31
32         float vibrateSpeed = 4.3f;
33         float vibratoSize = 15.0f;
34         float vibratoNow = frequency * (pow(2, vibratoSize / 1200.0f) - 1.0f);
35
36         Oscil vibrato = new Oscil(vibrateSpeed, vibratoNow, Waves.SINE);
37         vibrato.offset.setLastValue(frequency);
38         vibrato.patch(tone.frequency);
39
40         MoogFilter toneLPF = new MoogFilter(cutoff, 0.0f, MoogFilter.Type.LP);
41
42         Noise breath = new Noise(1.0f, Noise.Tint.WHITE);
43         MoogFilter breathBand = new MoogFilter(2000.0f, 0.15f, MoogFilter.Type.BP);
44
45         toneEnv = new ADSR(maxAmp, 0.08f, 0.14f, 0.82f, 0.16f);
46         breathEnv = new ADSR(maxAmp * 0.14f, 0.015f, 0.12f, 0.04f, 0.10f);
47
48         Noise breath2 = new Noise(1.0f, Noise.Tint.WHITE);
49         MoogFilter breathBand2 = new MoogFilter(700.0f, 0.2f, MoogFilter.Type.BP);
50         breathEnv2 = new ADSR(maxAmp * 0.11f, 0.015f, 0.12f, 0.07f, 0.10f);
51
52
53         tone.patch(toneLPF);
54         toneLPF.patch(toneEnv);
55
56         breath.patch(breathBand);
57         breathBand.patch(breathEnv);

```

```

58
59     breath2.patch(breathBand2);
60     breathBand2.patch(breathEnv2);
61 }
62
63 void noteOn(float duration)
64 {
65     toneEnv.patch(out);
66     breathEnv.patch(out);
67     breathEnv2.patch(out);
68
69     toneEnv.unpatchAfterRelease(out);
70     breathEnv.unpatchAfterRelease(out);
71     breathEnv2.unpatchAfterRelease(out);
72
73     toneEnv.noteOn();
74     breathEnv.noteOn();
75     breathEnv2.noteOn();
76 }
77
78 void noteOff()
79 {
80     toneEnv.noteOff();
81     breathEnv.noteOff();
82     breathEnv2.noteOff();
83 }
84 }
85
86
87 void setup()
88 {
89     size(512, 220);
90     minim = new Minim(this);
91     out = minim.getLineOut();
92     out.setTempo(120);
93
94     currentWaveform = makeFluteWaveform();
95 }
96
97 Wavetable makeFluteWaveform()
98 {
99     Wavetable wf = WavetableGenerator.gen10(
100         8192,
101         new float[] {
102             1.00f, 0.35f, 0.14f, 0.12f,
103             0.08f, 0.04f, 0.025f, 0.02f, 0.015f, 0.010f
104         }
105     );
106
107     wf.normalize();
108     wf.scale(0.75f);
109     return wf;
110 }
111

```

```

112 void playSong()
113 {
114     out.pauseNotes();
115
116     for (int i = 0; i < melody.length; i++)
117     {
118         out.playNote(
119             startTime[i],
120             duration[i],
121             new FluteInstrument(
122                 Frequency.ofPitch( melody[i] ).asHz(), 0.20f, currentWaveform)
123             );
124     }
125
126     out.resumeNotes();
127 }
128
129 void draw()
130 {
131     background(0);
132     stroke(255);
133
134     for (int i = 0; i < out.bufferSize() - 1; i++)
135     {
136         line(i, 70 - out.left.get(i) * 60,
137             i + 1, 70 - out.left.get(i + 1) * 60);
138
139         line(i, 160 - out.right.get(i) * 60,
140             i + 1, 160 - out.right.get(i + 1) * 60);
141     }
142
143     fill(255);
144
145 }
146
147 void keyPressed()
148 {
149     switch(key)
150     {
151         case 'p':
152             playSong();
153             break;
154
155         case '1':
156             currentWaveform = makeFluteWaveform();
157             break;
158     }
159 }
160
161
162 void stop()
163 {
164     minim.stop();
165     super.stop();

```

166 }

Listing 3 simpleI2C/master.ino

```
1 #include <Wire.h>
2
3 void setup()
4 {
5     Wire.begin();
6     Serial.begin(115200);
7 }
8
9 void loop()
10 {
11     Wire.beginTransmission(0x11);
12     Wire.write(0x01);
13     Wire.endTransmission();
14
15
16     Wire.requestFrom(0x11,1,true);
17
18     byte val = Wire.read();
19     Serial.println(val);
20
21     delay(500);
22
23
24 }
```

Listing 4 simpleI2C/slave.ino

```
1 #include <Wire.h>
2
3 byte SlaveADR = 0x11;
4
5 volatile byte receiveData = 0;
6 volatile bool newData = false;
7
8 void setup()
9 {
10     Wire.begin(SlaveADR);
11     Wire.onReceive(receiveEvent);
12     Wire.onRequest(requestEvent);
13     Serial.begin(115200);
14 }
15
16 void loop()
17 {
18     if(newData == true) {
19         Serial.println(receiveData);
20         newData = false;
21     }
22     delay(100);
```

```

23
24 }
25
26 void receiveEvent(int howMany){
27     while(Wire.available()){
28         receiveData = Wire.read();
29         newData = true;
30     }
31 }
32
33 void requestEvent(){
34     Wire.write(receiveData);
35 }

```

Listing 5 sync-I2C/master.ino

```

1 #include <Wire.h>
2
3 byte SlaveADR = 0x10;
4
5 volatile byte receiveData = 0;
6 volatile bool newData = false;
7
8 void setup()
9 {
10     Wire.begin(SlaveADR);
11     Wire.onReceive(receiveEvent);
12     Wire.onRequest(requestEvent);
13     Serial.begin(115200);
14 }
15
16 void loop()
17 {
18     if(newData == true) {
19         Serial.println(receiveData);
20         newData = false;
21     }
22     delay(100);
23 }
24
25
26 void receiveEvent(int howMany){
27     while(Wire.available()){
28         receiveData = Wire.read();
29         newData = true;
30     }
31 }
32
33 void requestEvent(){
34     Wire.write(receiveData);
35 }

```

```

1 #include <Wire.h>
2
3 const byte SlaveADR = 0x11;
4
5 const byte CMD_SYNC = 0x01;
6 const byte CMD_START = 0x10;
7 const byte CMD_CONFIG = 0x11;
8
9 volatile byte receivedCmd = 0;
10 volatile bool newData = false;
11
12 volatile uint16_t receivedBar = 0;
13 volatile uint16_t receivedBpmX10 = 0;
14
15 volatile byte receivedEntryOffset = 0;
16 volatile byte receivedLoopLength = 0;
17 volatile byte receivedPartId = 0;
18
19 void setup()
20 {
21     Wire.begin(SlaveADR);
22     Wire.onReceive(receiveEvent);
23
24     Serial.begin(115200);
25 }
26
27 void loop()
28 {
29     if(newData) {
30         noInterrupts();
31
32         byte cmd = receivedCmd;
33         uint16_t bar = receivedBar;
34         uint16_t bpmX10 = receivedBpmX10;
35         byte entryOffset = receivedEntryOffset;
36         byte loopLength = receivedLoopLength;
37         byte partId = receivedPartId;
38
39         newData = false;
40
41         interrupts();
42
43         if(cmd == CMD_SYNC){
44             Serial.println(bar);
45             Serial.println(bpmX10 / 10.0);
46         }else if(cmd == CMD_START){
47             Serial.println("START");
48         }else if(cmd == CMD_CONFIG){
49             Serial.println(entryOffset);
50             Serial.println(loopLength);
51             Serial.println(partId);
52         }

```

```

53     }
54 }
55
56 void receiveEvent(int howMany)
57 {
58     if (howMany < 1) {
59         return;
60     }
61
62     byte cmd = Wire.read();
63
64     if (cmd == CMD_SYNC && howMany == 5) {
65         byte barHigh = Wire.read();
66         byte barLow  = Wire.read();
67         byte bpmHigh = Wire.read();
68         byte bpmLow  = Wire.read();
69
70         receivedCmd = cmd;
71         receivedBar = ((uint16_t)barHigh << 8) | barLow;
72         receivedBpmX10 = ((uint16_t)bpmHigh << 8) | bpmLow;
73         newData = true;
74     }
75     else if (cmd == CMD_START && howMany == 1) {
76         receivedCmd = cmd;
77         newData = true;
78     }
79     else if (cmd == CMD_CONFIG && howMany == 4) {
80         receivedCmd = cmd;
81         receivedEntryOffset = Wire.read();
82         receivedLoopLength = Wire.read();
83         receivedPartId = Wire.read();
84         newData = true;
85     }
86     else {
87         while (Wire.available()) {
88             Wire.read();
89         }
90     }
91 }

```

参考文献

- [1] 「Arduino を 30 分で I2C 通信する」, Qiita, <https://qiita.com/MergeCells/items/20c3c1a0adfb222a19cd>, 2026 年 6 月 9 日閲覧.
- [2] 「【電子工作】複数の Arduino を接続しシリアル通信で制御してみる!」, ぶらり@ web 走り書き, <https://burariweb.info/electronic-work/multiple-arduino-connections-serial-communication.html>, 2026 年 6 月 9 日閲覧.